Supabase REST API and pgvector Operations: OpenAPI Schema Guide

Supabase provides a RESTful API built on PostgREST that auto-generates endpoints from your database schema, but vector similarity operators cannot be used directly in REST URLs—instead, AI operations require PostgreSQL RPC functions accessed via `/rest/v1/rpc/` endpoints. Supabase Supabase The API uses standard HTTP methods Supabase with specialized `Prefer` headers for controlling behavior, Apidog Supabase while Edge Functions at `/functions/v1/` provide serverless compute for embedding generation and AI workflows. All endpoints require an `apikey` header and use `Authorization: Bearer` tokens for authentication, Apidog Supabase with vector operations typically executed through custom SQL functions that wrap pgvector distance operators (`<=>`, `<#>`, `<->`).

Critical architectural constraints for AI operations

PostgREST does not support pgvector's similarity operators (`<=>`, `<#>`, `<->`) as URL query parameters, making RPC functions mandatory for vector search. Supabase supabase This architectural limitation means your OpenAPI schema must define vector operations as POST requests to `/rest/v1/rpc/{function_name}` Supabase rather than as query parameters on table endpoints. The inner product operator (`<#>`) delivers the fastest performance for normalized embeddings like OpenAI's, while cosine distance (`<=>`) returns distance values (0=identical) requiring conversion to similarity scores via `1 - distance`. github Swizec Teller

Vector search functions must order results by the distance operator directly (not computed columns) to utilize HNSW or IVFFlat indexes. For production deployments, HNSW indexes with `vector_ip_ops` provide superior query performance for normalized vectors, supporting up to 2,000 dimensions for standard vectors and 4,000 for halfvec types. github

Base URL structure and versioning

Production Base URL Pattern:

```
https://{project-ref}.supabase.co
```

API Endpoints:

- REST API: `/rest/v1/`
- Auth API: `/auth/v1/`
- Edge Functions: `/functions/v1/`

The `{project-ref}` is your unique project identifier found in Dashboard → Settings → API. All REST and Auth endpoints use v1 versioning, with no v2 available as of November 2025.

Required headers across all endpoints

Essential Headers:

```yaml
headers:
  apikey:
    description: Project API key (anon or service_role)
    required: true
    schema:
      type: string
    example: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  Authorization:
    description: Bearer token (anon key for RLS, user JWT for authenticated, service_role for admin)
    required: true
    schema:
      type: string
    example: "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  Content-Type:
    description: Request content type for POST/PATCH/PUT
    required: false
    schema:
      type: string
      enum: [application/json, application/x-www-form-urlencoded]
```

Prefer Header Values:

```yaml
Prefer:
  description: Controls API behavior and response format
  required: false
  schema:
    type: string
    enum:
      - "return=representation"  # Returns full inserted/updated rows
      - "return=minimal"         # Returns no data (default)
      - "return=headers-only"    # Returns only Location header
      - "count=exact"            # Returns exact row count (slow)
      - "count=planned"          # Returns planner estimate (fast)
      - "count=estimated"        # Returns pg_class estimate (fastest)
      - "resolution=merge-duplicates"   # UPSERT behavior
      - "resolution=ignore-duplicates"  # Skip conflicts
      - "params=single-object"   # Pass entire JSON as single RPC parameter
```

Database table operations (CRUD) endpoints

Endpoint Pattern:

```
{base_url}/rest/v1/{table_name}
```

SELECT operation

yaml

```
/rest/v1/{table_name}:
  get:
    summary: Query table rows
    parameters:
      - name: select
        in: query
        schema:
          type: string
        example: "id,title,content"
        description: Column names to return (comma-separated)
      - name: {column}
        in: query
        schema:
          type: string
        examples:
          equality: "eq.value"
          greater_than: "gt.25"
          less_than: "lt.100"
          like: "like.*pattern*"
          in_list: "in.(value1,value2,value3)"
          is_null: "is.null"
          not_null: "not.is.null"
      - name: order
        in: query
        schema:
          type: string
        examples:
          ascending: "column.asc"
          descending: "column.desc"
      - name: limit
        in: query
        schema:
          type: integer
        example: 10
      - name: offset
        in: query
        schema:
          type: integer
        example: 20
    responses:
      200:
        description: Successful query
        content:
```

```
application/json:
  schema:
    type: array
    items:
      type: object
```

PostgREST Query Operators:

Operator	Format	Description	Example
eq	?column=eq.value	Equals	?age=eq.25
neq	?column=neq.value	Not equal	?status=neq.deleted
gt	?column=gt.value	Greater than	?price=gt.100
gte	?column=gte.value	Greater than or equal	?age=gte.18
lt	?column=lt.value	Less than	?stock=lt.10
lte	?column=lte.value	Less than or equal	?discount=lte.50
like	?column=like.*pattern*	Pattern match (case-sensitive)	?name=like.*John*
ilike	?column=ilike.*pattern*	Pattern match (case-insensitive)	?email=ilike.*@gmail.com
in	?column=in.(val1,val2)	IN list	?status=in.(active,pending)
is	?column=is.null	IS comparison	?deleted=is.null
not	?column=not.eq.value	Negate any operator	?age=not.lt.18

INSERT operation

yaml

```yaml
/rest/v1/{table_name}:
  post:
    summary: Insert new rows
    requestBody:
      required: true
      content:
        application/json:
          schema:
            oneOf:
              - type: object
              - type: array
                items:
                  type: object
          examples:
            single_row:
              value:
                title: "New Document"
                content: "Document content"
            multiple_rows:
              value:
                - title: "Doc 1"
                  content: "Content 1"
                - title: "Doc 2"
                  content: "Content 2"
    parameters:
      - name: Prefer
        in: header
        schema:
          type: string
          default: "return=representation"
    responses:
      201:
        description: Successfully created
        headers:
          Location:
            schema:
              type: string
            description: URL of created resource
        content:
          application/json:
            schema:
              oneOf:
```

```
            - type: object
            - type: array
```

UPDATE operation

yaml

```yaml
/rest/v1/{table_name}:
  patch:
    summary: Update existing rows (requires filters)
    parameters:
      - name: {column}
        in: query
        required: true
        schema:
          type: string
        description: Filter to identify rows to update (e.g., id=eq.1)
      - name: Prefer
        in: header
        schema:
          type: string
          default: "return=representation"
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
          example:
            status: "completed"
            updated_at: "2025-11-12T10:00:00Z"
    responses:
      200:
        description: Successfully updated
```

DELETE operation

yaml

```yaml
/rest/v1/{table_name}:
  delete:
    summary: Delete rows (requires filters)
    parameters:
      - name: {column}
        in: query
        required: true
        schema:
          type: string
        description: Filter to identify rows to delete (e.g., id=eq.1)
    responses:
      204:
        description: Successfully deleted
```

UPSERT operation

yaml

```yaml
/rest/v1/{table_name}:
  post:
    summary: Insert or update on conflict
    parameters:
      - name: on_conflict
        in: query
        schema:
          type: string
        description: Column name(s) for conflict resolution
        example: "email"
      - name: Prefer
        in: header
        schema:
          type: string
          enum:
            - "resolution=merge-duplicates"
            - "resolution=ignore-duplicates"
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: array
```

RPC function calls for vector operations

Endpoint Pattern:

```
{base_url}/rest/v1/rpc/{function_name}
```

This is the **critical endpoint for all AI/vector operations** since pgvector operators cannot be used in standard table queries. Supabase

Vector similarity search RPC endpoint

yaml

```yaml
/rest/v1/rpc/match_documents:
  post:
    summary: Perform vector similarity search
    operationId: matchDocuments
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - query_embedding
            properties:
              query_embedding:
                type: array
                items:
                  type: number
                description: Query vector embedding
                example: [0.1, 0.2, 0.3, "...1536 values..."]
                minItems: 384
                maxItems: 1536
              match_threshold:
                type: number
                format: float
                description: Minimum similarity score (0-1)
                default: 0.78
                minimum: 0
                maximum: 1
              match_count:
                type: integer
                description: Maximum number of results to return
                default: 10
                minimum: 1
                maximum: 200
    responses:
      200:
        description: Matching documents with similarity scores
        content:
          application/json:
            schema:
              type: array
              items:
                type: object
```

```
    properties:
      id:
        type: integer
      title:
        type: string
      content:
        type: string
      similarity:
        type: number
        format: float
        description: Similarity score (0-1, higher is more similar)
  example:
    - id: 42
      title: "Vector Search Guide"
      content: "Complete guide to pgvector..."
      similarity: 0.89
    - id: 17
      title: "AI Operations"
      content: "Using embeddings for search..."
      similarity: 0.85
```

Corresponding SQL Function:

```sql
```

```
CREATE OR REPLACE FUNCTION match_documents (
  query_embedding vector(1536),
  match_threshold FLOAT DEFAULT 0.78,
  match_count INT DEFAULT 10
)
RETURNS TABLE (
  id BIGINT,
  title TEXT,
  content TEXT,
  similarity FLOAT
)
LANGUAGE plpgsql
AS $$
BEGIN
  RETURN QUERY
  SELECT
    documents.id,
    documents.title,
    documents.content,
    1 - (documents.embedding <=> query_embedding) AS similarity
  FROM documents
  WHERE 1 - (documents.embedding <=> query_embedding) > match_threshold
  ORDER BY documents.embedding <=> query_embedding ASC
  LIMIT match_count;
END;
$$;
```

Hybrid search RPC endpoint

yaml

```yaml
/rest/v1/rpc/hybrid_search:
  post:
    summary: Combined full-text and vector similarity search
    operationId: hybridSearch
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - query_text
              - query_embedding
            properties:
              query_text:
                type: string
                description: Text query for full-text search
                example: "machine learning embeddings"
              query_embedding:
                type: array
                items:
                  type: number
                description: Query vector for semantic search
              match_count:
                type: integer
                default: 10
              full_text_weight:
                type: number
                format: float
                default: 1.0
                description: Weight for full-text results in RRF
              semantic_weight:
                type: number
                format: float
                default: 1.0
                description: Weight for semantic results in RRF
              rrf_k:
                type: integer
                default: 50
                description: RRF constant parameter
    responses:
      200:
        description: Combined search results with RRF ranking
```

```
    content:
      application/json:
        schema:
          type: array
          items:
            type: object
```

General RPC endpoint specification

yaml

```
/rest/v1/rpc/{function_name}:
  post:
    summary: Call PostgreSQL function via RPC
    operationId: callRpcFunction
    parameters:
      - name: function_name
        in: path
        required: true
        schema:
          type: string
        description: Name of the PostgreSQL function to call
      - name: Prefer
        in: header
        schema:
          type: string
          enum:
            - "params=single-object"
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            description: Parameters matching function signature
            additionalProperties: true
    responses:
      200:
        description: Function result
        content:
          application/json:
            schema:
              oneOf:
                - type: object
                - type: array
                - type: number
                - type: string
                - type: boolean
```

Vector similarity operators and query patterns

pgvector Distance Operators:

Operator	Type	Formula	Use Case	Index
`<=>`	Cosine distance	1 - (A·B)/(‖A‖‖B‖)	Default safe choice	`vector_cosine_ops`
`<#>`	Negative inner product	-(A·B)	**Fastest for normalized vectors**	`vector_ip_ops`
`<->`	Euclidean distance (L2)	√Σ(Ai-Bi)²	Raw distance measurement	`vector_l2_ops`

Critical Implementation Notes:

1. **Cosine distance returns distance (not similarity)**: Convert to similarity using `1 - (embedding <=> query)` github
2. **Order by distance operator directly**: `ORDER BY embedding <=> query` uses index; `ORDER BY similarity DESC` may not
3. **Use inner product for OpenAI embeddings**: OpenAI normalizes embeddings, making `<#>` fastest github
4. **HNSW indexes recommended**: Better performance than IVFFlat for production github

SQL Index Creation:

```sql
-- HNSW index (recommended for production)
CREATE INDEX ON documents
USING hnsw (embedding vector_ip_ops)
WITH (m = 16, ef_construction = 64);

-- For cosine distance
CREATE INDEX ON documents
USING hnsw (embedding vector_cosine_ops);

-- IVFFlat alternative (faster build, lower query performance)
CREATE INDEX ON documents
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

Authentication API endpoints

Base Path: `/auth/v1/`

Sign up endpoint

yaml

```yaml
/auth/v1/signup:
  post:
    summary: Create new user account
    operationId: signUp
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - email
              - password
            properties:
              email:
                type: string
                format: email
              password:
                type: string
                format: password
                minLength: 6
              data:
                type: object
                description: User metadata
                additionalProperties: true
          example:
            email: "user@example.com"
            password: "secure-password"
            data:
              first_name: "John"
              age: 30
    responses:
      200:
        description: User created successfully
        content:
          application/json:
            schema:
              type: object
              properties:
                access_token:
                  type: string
                refresh_token:
                  type: string
```

```
    expires_in:
      type: integer
    user:
      type: object
```

Sign in endpoint

yaml

```yaml
/auth/v1/token:
  post:
    summary: Sign in with email and password
    operationId: signIn
    parameters:
      - name: grant_type
        in: query
        required: true
        schema:
          type: string
          enum: [password, refresh_token]
    requestBody:
      required: true
      content:
        application/json:
          schema:
            oneOf:
              - type: object
                title: Password Grant
                required:
                  - email
                  - password
                properties:
                  email:
                    type: string
                    format: email
                  password:
                    type: string
              - type: object
                title: Refresh Token Grant
                required:
                  - refresh_token
                properties:
                  refresh_token:
                    type: string
    responses:
      200:
        description: Authentication successful
        content:
          application/json:
            schema:
              type: object
              properties:
```

```yaml
    access_token:
      type: string
      description: JWT token for authenticated requests
    refresh_token:
      type: string
    expires_in:
      type: integer
      description: Token expiration in seconds
    user:
      type: object
```

User management endpoints

```yaml
```

```
/auth/v1/user:
  get:
    summary: Get current user
    operationId: getUser
    security:
      - BearerAuth: []
    responses:
      200:
        description: Current user details
        content:
          application/json:
            schema:
              type: object

  put:
    summary: Update user details
    operationId: updateUser
    security:
      - BearerAuth: []
    requestBody:
      content:
        application/json:
          schema:
            type: object
            properties:
              email:
                type: string
                format: email
              password:
                type: string
              data:
                type: object
    responses:
      200:
        description: User updated successfully

/auth/v1/logout:
  post:
    summary: Sign out user
    operationId: signOut
    security:
      - BearerAuth: []
    responses:
```

```
204:
  description: Successfully signed out
```

Edge Functions API for AI operations

Base Path: `/functions/v1/`

Endpoint Pattern:

```
{base_url}/functions/v1/{function_name}
```

Embedding generation function

yaml

```yaml
/functions/v1/generate-embeddings:
  post:
    summary: Generate text embeddings using built-in gte-small model
    operationId: generateEmbeddings
    security:
      - ApiKeyAuth: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - input
            properties:
              input:
                type: string
                description: Text to generate embedding for
                maxLength: 512
                example: "Machine learning and AI documentation"
    responses:
      200:
        description: Generated embedding vector
        content:
          application/json:
            schema:
              type: object
              properties:
                embedding:
                  type: array
                  items:
                    type: number
                  description: 384-dimensional embedding vector (gte-small)
                  minItems: 384
                  maxItems: 384
```

Corresponding Edge Function Implementation:

typescript

```
import 'jsr:@supabase/functions-js/edge-runtime.d.ts'

const model = new Supabase.ai.Session('gte-small')

Deno.serve(async (req: Request) => {
  const { input } = await req.json()

  const embedding = await model.run(input, {
    mean_pool: true,
    normalize: true
  })

  return new Response(JSON.stringify({ embedding }), {
    headers: { 'Content-Type': 'application/json' }
  })
})
```

Semantic search function

yaml

```yaml
/functions/v1/semantic-search:
  post:
    summary: Perform end-to-end semantic search
    operationId: semanticSearch
    security:
      - ApiKeyAuth: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - query
            properties:
              query:
                type: string
                description: Search query
              match_threshold:
                type: number
                default: 0.8
              match_count:
                type: integer
                default: 10
    responses:
      200:
        description: Search results with similarity scores
        content:
          application/json:
            schema:
              type: object
              properties:
                query:
                  type: string
                results:
                  type: array
                  items:
                    type: object
```

Edge Function Implementation:

```typescript
```

```
import { createClient } from "jsr:@supabase/supabase-js@2"

const supabase = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
)

const model = new Supabase.ai.Session("gte-small")

Deno.serve(async (req) => {
  const { query, match_threshold = 0.8, match_count = 10 } = await req.json()

  // Generate embedding
  const embedding = await model.run(query, {
    mean_pool: true,
    normalize: true
  })

  // Query database
  const { data: results, error } = await supabase
    .rpc("match_documents", {
      query_embedding: JSON.stringify(embedding),
      match_threshold,
      match_count
    })

  if (error) return Response.json({ error: error.message }, { status: 500 })

  return Response.json({ query, results })
})
```

LLM streaming function

```
yaml
```

```
/functions/v1/chat-completion:
  post:
    summary: Generate streaming LLM responses
    operationId: chatCompletion
    security:
      - ApiKeyAuth: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - prompt
            properties:
              prompt:
                type: string
              model:
                type: string
                default: "mistral"
              stream:
                type: boolean
                default: true
    responses:
      200:
        description: Streaming LLM response
        content:
          text/event-stream:
            schema:
              type: string
```

Complete examples and workflows

Example 1: Creating a vector-enabled table

SQL Schema:

```
sql
```

```sql
-- Enable pgvector extension
CREATE EXTENSION IF NOT EXISTS vector WITH SCHEMA extensions;

-- Create documents table with vector column
CREATE TABLE documents (
  id BIGSERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  content TEXT NOT NULL,
  metadata JSONB,
  embedding vector(1536),  -- OpenAI text-embedding-3-small
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Create HNSW index for fast similarity search
CREATE INDEX ON documents
USING hnsw (embedding vector_ip_ops)
WITH (m = 16, ef_construction = 64);

-- Enable Row Level Security
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;

-- Create policy for authenticated users
CREATE POLICY "Users can view own documents"
  ON documents FOR SELECT
  USING (auth.uid() = user_id);
```

Supabase

Example 2: REST API request for inserting document with embedding

HTTP Request:

```http
```

```
POST https://project-ref.supabase.co/rest/v1/documents
Content-Type: application/json
apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Prefer: return=representation

{
  "title": "Vector Search Guide",
  "content": "Comprehensive guide to using pgvector with Supabase...",
  "embedding": [0.023, -0.015, 0.042, "...1533 more values...", 0.018]
}
```

Response:

```
json

[
  {
    "id": 42,
    "title": "Vector Search Guide",
    "content": "Comprehensive guide to using pgvector with Supabase...",
    "metadata": null,
    "embedding": "[0.023,-0.015,0.042,...]",
    "created_at": "2025-11-12T10:30:00Z"
  }
]
```

Example 3: Vector similarity search via RPC

HTTP Request:

```
http

POST https://project-ref.supabase.co/rest/v1/rpc/match_documents
Content-Type: application/json
apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

{
  "query_embedding": [0.021, -0.013, 0.045, "...1533 more values...", 0.016],
  "match_threshold": 0.78,
  "match_count": 5
}
```

Response:

```json
[
  {
    "id": 42,
    "title": "Vector Search Guide",
    "content": "Comprehensive guide to using pgvector...",
    "similarity": 0.94
  },
  {
    "id": 17,
    "title": "AI Operations Manual",
    "content": "Using embeddings for semantic search...",
    "similarity": 0.87
  },
  {
    "id": 93,
    "title": "Database Vectors",
    "content": "Storing and querying vector embeddings...",
    "similarity": 0.82
  }
]
```

Example 4: Complete RAG workflow with JavaScript

```javascript
```

```
import { createClient } from '@supabase/supabase-js'
import OpenAI from 'openai'

const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_ROLE_KEY
)

const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY
})

// Step 1: Generate embedding for user query
async function searchAndAnswer(query) {
  // Generate query embedding
  const embeddingResponse = await openai.embeddings.create({
    model: 'text-embedding-3-small',
    input: query,
    dimensions: 1536
  })

  const [{ embedding }] = embeddingResponse.data

  // Step 2: Search for similar documents
  const { data: documents, error } = await supabase
    .rpc('match_documents', {
      query_embedding: embedding,
      match_threshold: 0.78,
      match_count: 5
    })

  if (error) throw error

  // Step 3: Build context from results
  const context = documents
    .map(doc => `${doc.title}\n\n${doc.content}`)
    .join('\n\n---\n\n')

  // Step 4: Generate AI response with context
  const completion = await openai.chat.completions.create({
    model: 'gpt-4',
    messages: [
      {
```

```
      role: 'system',
      content: `Answer the user's question based on this context:\n\n${context}`
    },
    {
      role: 'user',
      content: query
    }
  ]
 })

 return completion.choices[0].message.content
}

// Usage
const answer = await searchAndAnswer("How do I use pgvector with Supabase?")
console.log(answer)
```

Vercel

Example 5: Hybrid search implementation

SQL Function:

sql

```sql
CREATE OR REPLACE FUNCTION hybrid_search(
  query_text TEXT,
  query_embedding vector(512),
  match_count INT DEFAULT 10,
  full_text_weight FLOAT DEFAULT 1,
  semantic_weight FLOAT DEFAULT 1,
  rrf_k INT DEFAULT 50
)
RETURNS TABLE (
  id BIGINT,
  title TEXT,
  content TEXT,
  rrf_score FLOAT
)
LANGUAGE sql
AS $$
WITH full_text AS (
  SELECT
    id,
    ROW_NUMBER() OVER(ORDER BY ts_rank_cd(fts, websearch_to_tsquery(query_text)) DESC) AS rank_ix
  FROM documents
  WHERE fts @@ websearch_to_tsquery(query_text)
  ORDER BY rank_ix
  LIMIT LEAST(match_count, 30) * 2
),
semantic AS (
  SELECT
    id,
    ROW_NUMBER() OVER (ORDER BY embedding <#> query_embedding) AS rank_ix
  FROM documents
  ORDER BY rank_ix
  LIMIT LEAST(match_count, 30) * 2
)
SELECT
  documents.id,
  documents.title,
  documents.content,
  COALESCE(1.0 / (rrf_k + full_text.rank_ix), 0.0) * full_text_weight +
  COALESCE(1.0 / (rrf_k + semantic.rank_ix), 0.0) * semantic_weight AS rrf_score
FROM full_text
FULL OUTER JOIN semantic ON full_text.id = semantic.id
JOIN documents ON COALESCE(full_text.id, semantic.id) = documents.id
ORDER BY rrf_score DESC
```

```
LIMIT match_count
$$;
```

JavaScript Client Call:

```javascript
const { data: results } = await supabase.rpc('hybrid_search', {
  query_text: 'vector search',
  query_embedding: embedding,
  match_count: 10,
  full_text_weight: 1.0,
  semantic_weight: 1.5  // Prioritize semantic matches
})
```

OpenAPI schema considerations

Security schemes

```yaml
components:
  securitySchemes:
    ApiKeyAuth:
      type: apiKey
      in: header
      name: apikey
      description: Supabase project API key (anon or service_role)

    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
      description: User access token from authentication

security:
  - ApiKeyAuth: []
  - BearerAuth: []
```

Reusable schemas

```yaml
```

```yaml
components:
  schemas:
    Vector1536:
      type: array
      items:
        type: number
        format: float
      minItems: 1536
      maxItems: 1536
      description: OpenAI text-embedding-3-small vector

    Vector384:
      type: array
      items:
        type: number
      minItems: 384
      maxItems: 384
      description: Supabase gte-small vector

    MatchDocumentsRequest:
      type: object
      required:
        - query_embedding
      properties:
        query_embedding:
          oneOf:
            - $ref: '#/components/schemas/Vector1536'
            - $ref: '#/components/schemas/Vector384'
        match_threshold:
          type: number
          format: float
          minimum: 0
          maximum: 1
          default: 0.78
        match_count:
          type: integer
          minimum: 1
          maximum: 200
          default: 10

    DocumentResult:
      type: object
      properties:
```

```yaml
      id:
        type: integer
        format: int64
      title:
        type: string
      content:
        type: string
      similarity:
        type: number
        format: float
        minimum: 0
        maximum: 1
      metadata:
        type: object
        additionalProperties: true

  ErrorResponse:
    type: object
    properties:
      message:
        type: string
      code:
        type: string
      details:
        type: string
```

Server configuration

yaml

```yaml
servers:
  - url: https://{project-ref}.supabase.co
    description: Production Supabase project
    variables:
      project-ref:
        default: your-project-ref
        description: Your unique Supabase project reference ID

  - url: http://localhost:54321
    description: Local development server
```

Parameter definitions

```yaml
components:
  parameters:
    TableName:
      name: table_name
      in: path
      required: true
      schema:
        type: string
      description: Database table name

    FunctionName:
      name: function_name
      in: path
      required: true
      schema:
        type: string
      description: PostgreSQL function name

    PreferHeader:
      name: Prefer
      in: header
      required: false
      schema:
        type: string
      description: PostgREST Prefer header for controlling behavior

    SelectColumns:
      name: select
      in: query
      required: false
      schema:
        type: string
      description: Comma-separated list of columns to return
      example: "id,title,content"
```

Key implementation patterns for OpenAPI specs

Pattern 1: RPC endpoints must be defined individually since each function has unique parameters and return types. Auto-generation from database schema is not fully possible.

Pattern 2: Vector types should be defined as arrays with min/max item constraints matching your embedding dimensions (384, 512, 1536, etc.).

Pattern 3: All write operations (POST, PATCH, DELETE) should include the `Prefer: return=representation` header example to show users how to get response data.

Pattern 4: Query parameters for table endpoints use PostgREST operators (eq, gt, like, etc.) and should be documented with examples of each operator type.

Pattern 5: Edge Functions are separate HTTP endpoints with their own authentication and should be in a different OpenAPI path group from REST API endpoints.

Pattern 6: Authentication flows require documenting both the anon key (for client-side with RLS) and service_role key (for server-side admin operations) as separate security schemes.

Pattern 7: Response schemas should account for both single objects and arrays since Supabase can return either depending on the query.

This specification provides the foundation for creating a complete OpenAPI 3.1.0 schema for Supabase AI operations, with particular emphasis on the RPC-based approach required for vector similarity search and the architectural constraints of PostgREST's integration with pgvector.